War Card Game Simulator — Documentation

War Card Game Simulator — Documentation

Table of Contents

1. Project Overview
2. Technology Stack
3. Project Structure
4. Database Schema
5. Schema Notes & Assumptions
6. Game Rules Implemented
7. Backend API Reference
8. Design Decisions
9. Running Locally
10. Environment Variables
11. Running with Docker
12. Testing & Verification
13. Submission Zip Notes
14. Time-Box Note & Future Improvements

Project Overview

War Card Game Simulator is a full-stack implementation of the classic two-player card game *War*. It consists of:

- A **Python FastAPI** backend
- A **SQLite** database (with **Turso/libSQL** compatibility for production/cloud deployment)
- A **React + TypeScript** frontend

Core Capabilities

Feature	Description
New Game	Starts a new game with a freshly shuffled 52-card deck
Dealing	Deals 26 cards to each of the two players
Round-by-Round Play	Play a single round at a time via the UI or API
Full Simulation	Simulate the entire game to completion in one request
War Handling	Correctly resolves tie rounds (“War”) including nested/repeated wars
Persistence	Persists exact game state so a player can leave and resume later
Round Logs	Stores and displays a full history of rounds played
Analytics	Provides stats, leaderboards, and aggregate reporting
CSV Export	Frontend can export game/stat reports as CSV

Technology Stack

Backend

- Python 3.10+
- FastAPI
- Pydantic v2
- Uvicorn (ASGI server)
- SQLite (default, local fallback)

- Turso/libSQL support via environment variables

Frontend

- React 18
- TypeScript
- Tailwind CSS
- Axios (HTTP client)
- Recharts (charts/analytics visualization)
- Framer Motion (animations)
- React Hot Toast (notifications)

Database

- **Local development:** SQLite
- **Production/cloud:** Turso/libSQL-compatible schema (same schema works on both)

Project Structure

```
war-game/
  backend/
    app/
      main.py              # FastAPI app setup, CORS, startup, health checks
      database.py           # DB connection manager and schema initialization
      game_logic.py          # Core War simulator classes and game rules
      models.py              # Pydantic request/response models
      routes/
        games.py             # Game creation, rounds, simulation, state APIs
        stats.py              # Analytics and leaderboard APIs
    migrations/
      001_initial_schema.sql
    requirements.txt
    Dockerfile

  frontend/
    src/
      App.tsx
      pages/
        HomePage.tsx
        GamePage.tsx
        StatsPage.tsx
      components/
        CardAsset.tsx
        CardDisplay.tsx
        GameBoard.tsx
        GameController.tsx
        RoundHistory.tsx
        GameStats.tsx
      services/
        api.ts
        sound.ts
      types/
        index.ts
    package.json
    Dockerfile

  docker-compose.yml
  README.md
```

Backend Module Responsibilities

- **`main.py`** — Bootstraps the FastAPI app, configures CORS, registers routers, and exposes startup/health-check hooks.
- **`database.py`** — Manages the DB connection (SQLite or Turso/libSQL) and initializes the schema on startup.
- **`game_logic.py`** — Contains the core game engine: `Card`, `Deck`, `Player`, and `WarGame` classes that implement all War rules.
- **`models.py`** — Pydantic v2 models defining request and response shapes for the API.

- **`routes/games.py`** — Endpoints for creating games, playing rounds, simulating, and retrieving state.
- **`routes/stats.py`** — Endpoints for analytics, leaderboards, and aggregate statistics.

Frontend Module Responsibilities

- **`pages/`** — Top-level routed views: Home, active Game, and Stats dashboard.
- **`components/`** — Reusable UI building blocks (card rendering, game board, controls, round history, stats widgets).
- **`services/api.ts`** — Centralized, typed Axios wrapper; all HTTP calls go through this layer.
- **`services/sound.ts`** — Sound-effect handling for gameplay events.
- **`types/index.ts`** — Shared TypeScript type definitions used across components and services.

Database Schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS games (
    id TEXT PRIMARY KEY,
    start_time DATETIME NOT NULL,
    end_time DATETIME,
    winner_player INTEGER CHECK (winner_player IN (1, 2) OR winner_player IS NULL),
    total_rounds INTEGER DEFAULT 0,
    status TEXT DEFAULT 'in_progress' CHECK (status IN ('in_progress', 'completed', 'abandoned')),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS rounds (
    id TEXT PRIMARY KEY,
    game_id TEXT NOT NULL,
    round_number INTEGER NOT NULL,
    player1_card TEXT NOT NULL,
    player2_card TEXT NOT NULL,
    is_war BOOLEAN DEFAULT 0,
    war_round_count INTEGER DEFAULT 0,
    winner_player INTEGER CHECK (winner_player IN (1, 2) OR winner_player IS NULL),
    cards_won INTEGER DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS war_details (
    id TEXT PRIMARY KEY,
    round_id TEXT NOT NULL,
    war_round_number INTEGER NOT NULL,
    player1_card TEXT NOT NULL,
    player2_card TEXT NOT NULL,
    winner_player INTEGER CHECK (winner_player IN (1, 2) OR winner_player IS NULL),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (round_id) REFERENCES rounds(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS game_states (
    game_id TEXT PRIMARY KEY,
    player1_name TEXT NOT NULL DEFAULT 'Player 1',
    player2_name TEXT NOT NULL DEFAULT 'Player 2',
    player1_hand TEXT NOT NULL,
    player2_hand TEXT NOT NULL,
    round_counter INTEGER NOT NULL DEFAULT 0,
    total_wars INTEGER NOT NULL DEFAULT 0,
    max_rounds INTEGER NOT NULL DEFAULT 1000,
    status TEXT NOT NULL DEFAULT 'in_progress',
    winner_player INTEGER,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_rounds_game_id ON rounds(game_id);
CREATE INDEX IF NOT EXISTS idx_rounds_created_at ON rounds(created_at);
CREATE INDEX IF NOT EXISTS idx_war_details_round_id ON war_details(round_id);
CREATE INDEX IF NOT EXISTS idx_game_states_updated_at ON game_states(updated_at);
```

Table Summary

Table	Purpose	Key Relationships
`games`	Match-level metadata: start/end time, status, winner, total rounds	Parent of `rounds` and `game_states`
`rounds`	One row per top-level round played	`game_id` → `games.id` (CASCADE)
`war_details`	Nested tie-resolution steps for rounds that triggered War	`round_id` → `rounds.id` (CASCADE)
`game_states`	Exact remaining hands (as JSON) for resuming a game	`game_id` → `games.id` (CASCADE)

Schema Notes & Assumptions

- `games` stores match-level metadata (start/end time, status, final winner, round count).
- `rounds` stores **one top-level round per play** — this is the record created every time both players reveal a card, regardless of whether that round triggers a War.
- `war_details` stores **nested tie-resolution steps** for rounds that triggered War, allowing a full history of every card played during a multi-stage War.
- `game_states` stores the **exact remaining hands as JSON**, so a game can be resumed after a browser refresh or backend restart without losing state.
- `winner_player` is always `1`, `2`, or `NULL` (used for ties or games still in progress).
- Cards are stored in **short format**, e.g. `AS` (Ace of Spades), `10H` (Ten of Hearts), `KD` (King of Diamonds).

Game Rules Implemented

1. A standard 52-card deck is shuffled at the start of each game.
2. Player 1 and Player 2 each receive **26 cards**.
3. Each round, both players reveal their top card.
4. The **higher-ranked card wins** the round and its player collects both played cards.
5. Card ranking: **2 is lowest, Ace is highest**.
6. If both revealed cards have **equal rank**, a **War** is triggered.
7. During each War stage, **one additional face-up card** is played by each player to resolve the tie.
8. If the War card **also ties**, War repeats (nested War) until one player wins or a player cannot continue (runs out of cards).
9. The game ends when:
    - One player holds **all 52 cards**, or
    - A player **runs out of cards**, or
    - The **maximum round limit** is reached.
10. If the maximum round limit is reached, the player holding **more cards** at that point is declared the winner.

Backend API Reference

Game APIs

Method	Endpoint	Description
`POST`	`/api/games`	Create a new game (shuffles and deals a fresh deck)
`GET`	`/api/games`	List all games
`GET`	`/api/games/{game_id}`	Retrieve metadata for a specific game
`GET`	`/api/games/{game_id}/rounds`	Retrieve full round history for a game
`POST`	`/api/games/{game_id}/next-round`	Play a single round
`POST`	`/api/games/{game_id}/simulate`	Simulate the rest of the game to completion

POST	/api/games/{game_id}/complete	Manually mark/complete a game based on current card counts
GET	/api/games/{game_id}/state	Retrieve the exact current resumable state (hands, counters)

Stats APIs

Method	Endpoint	Description
GET	/api/stats/overview	Aggregate overview stats across all games
GET	/api/stats/player/{player_id}	Stats for an individual player
GET	/api/stats/most-wars	Games/players ranked by number of War events
GET	/api/stats/leaderboard	Leaderboard ranking
GET	/api/stats/recent	Most recently played/created games

System APIs

Method	Endpoint	Description
GET	/health	Health check endpoint
GET	/docs	Auto-generated Swagger/OpenAPI docs (interactive)
GET	/redoc	Auto-generated ReDoc API documentation

Design Decisions

- **Isolated game engine:** The core simulator logic lives entirely in `game_logic.py` using dedicated `Card`, `Deck`, `Player`, and `WarGame` classes.
- **Thin API routes:** API route handlers do **not** implement card rules directly — they call into the game engine and persist the results, keeping business logic decoupled from HTTP concerns.
- **Dual persistence model:** The database stores both an **event history** (rounds, war_details) and an **exact resumable state** (game_states), rather than trying to reconstruct state by replaying history.
- **Separation of history and state:** Round history is kept separate from current game state. This keeps analytics queries simple while still allowing exact game resumption.
- **Typed frontend API layer:** The frontend routes all HTTP calls through a single typed service (`services/api.ts`) so components never call Axios directly — this centralizes error handling and typing.
- **Independent data fetching:** The UI fetches game metadata, current state, and full round history as **separate calls**, rather than one large combined payload.
- **Full simulation persistence:** “Simulate” mode persists **every generated round**, not just the final result — preserving full analytics fidelity even for auto-played games.
- **Manual completion logic:** The manual “complete” action selects a winner based on **current card counts** at the time of completion.

Running Locally

Backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

On macOS/Linux, activate the virtual environment with `source venv/bin/activate` instead of `venv\Scripts\activate`.

Frontend

```
cd frontend
npm install
npm start
```

Access Points

Resource	URL
Frontend App	http://localhost:3000
Backend API	http://localhost:8000
Swagger Docs	http://localhost:8000/docs
ReDoc Docs	http://localhost:8000/redoc

Environment Variables

Backend (`.env`, optional)

```
PROJECT_NAME=War Card Game Simulator API
DATABASE_URL=sqlite:///./war_game.db
CORS_ORIGINS=["http://localhost:3000"]
DEBUG=True
```

For Turso/libSQL (cloud database):

```
TURSO_DATABASE_URL=libsql://your-database-url
TURSO_AUTH_TOKEN=your-token
CORS_ORIGINS=["http://localhost:3000"]
```

Frontend (`.env`, optional)

```
REACT_APP_API_URL=http://localhost:8000
```

Running with Docker

```
docker-compose up --build
```

Service	URL
Frontend	http://localhost:3000
Backend	http://localhost:8000

Testing & Verification

Automated Checks

```
# Backend — compile check
cd backend
python -m compileall -q app

# Frontend — production build
cd frontend
npm run build
```

Manual Verification Flow

1. Start both the backend and frontend servers.
2. Create a new game.
3. Confirm both players start with **26 cards** each.
4. Play a single round.
5. Confirm the round number increments and card counts update correctly.
6. Refresh the page (or leave and reopen the game URL).
7. Confirm the game **resumes from the same round** with the same card counts.
8. Click **Simulate All**.
9. Confirm the final result correctly shows: Player 1 wins / Player 2 wins / tie or War rounds, and the

declared final winner.

Submission Zip Notes

Include

- `backend/`
- `frontend/`
- `docker-compose.yml`
- `README.md` or `DOCUMENTATION.md`
- `DEPLOYMENT.md` (if desired)

Exclude

- `frontend/node_modules/`
- `frontend/build/`
- `backend/venv/`
- `backend/war_game.db`
- `__pycache__/`
- `*.pyc`

Time-Box Note & Future Improvements

This implementation was time-boxed to the expected **4–6 hour** exercise scope. The primary focus was on delivering:

- A working War game simulator
- Persistent, resumable game state
- A complete REST API
- Functional frontend gameplay
- Basic analytics

With More Time, Planned Improvements Include:

- Automated backend unit tests for deterministic War scenarios.
- Frontend component tests for card counts and final result rendering.
- A seed option for reproducible simulations.
- Better migration/version management, instead of a single initial SQL file.
- WebSocket updates for live, multi-tab game state synchronization.
- A cleaner packaging script for generating a submission zip automatically.
- More robust analytics — e.g., average War depth and longest tie chain.
- Authentication or named user accounts, if this evolved into a multi-user product.